Полиномиальная регрессия

Формулы, расчёт на пяти точках и Python

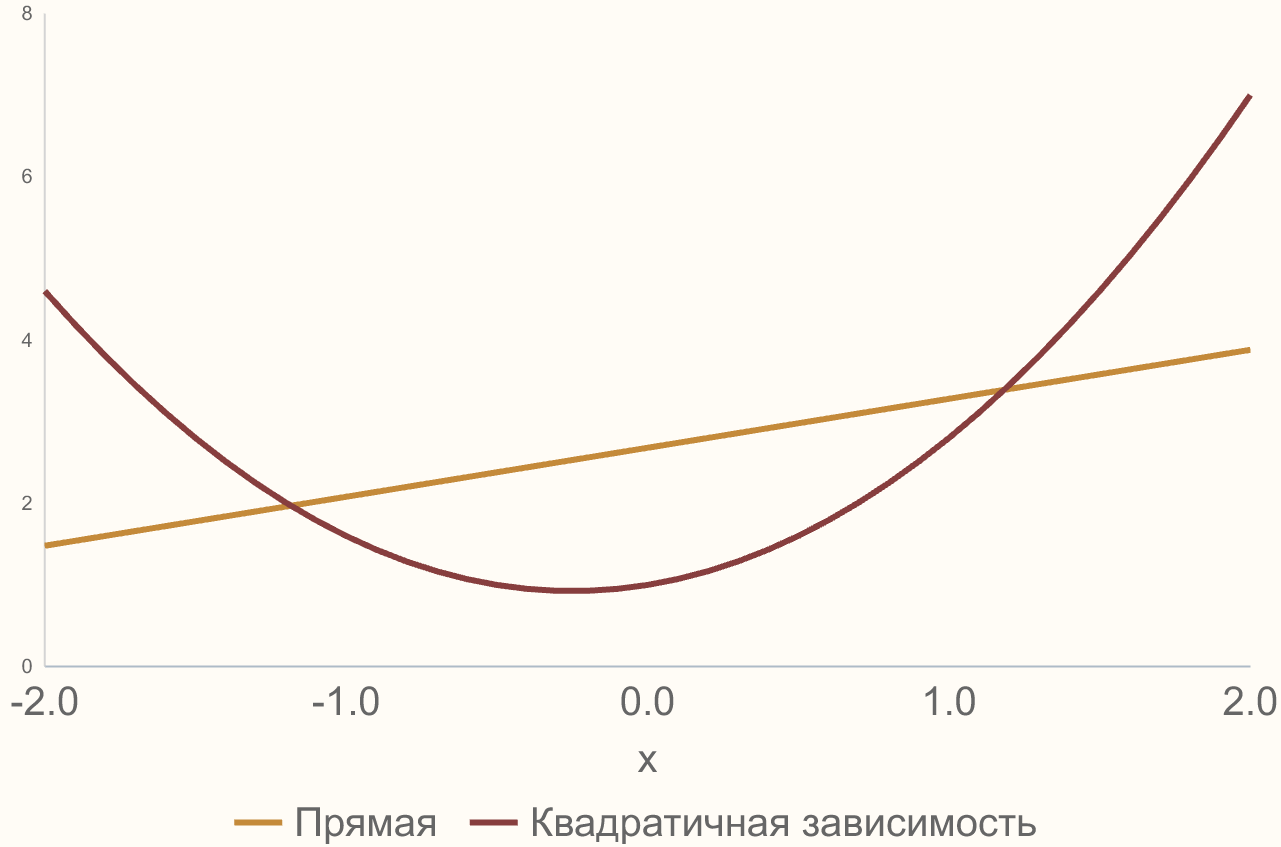
Савосина Дарья
Андреев Александр

Зачем нужен полином

$$y = f(x) + \varepsilon$$
$$E[\varepsilon \mid x] = 0$$

Прогноз числового отклика,
когда наклон зависимости
меняется

Примеры:
калибровка датчика
расход энергии при росте нагрузки



Пример на синтетических данных. Полином приближает зависимость в выбранном диапазоне.

Модель и базисные функции

$$\hat{y}(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta_d x^d$$

$$\varphi(x) = (1, x, x^2, \ldots, x^d)^T$$

$$\hat{y}(x) = \varphi(x)^T \beta$$

d задаёт степень полинома
β определяет кривую
Коэффициенты находим методами линейной регрессии

Несколько входных признаков

$p = 2, d = 2$: $1, x_1, x_2, x_1^2, x_1x_2, x_2^2$

Член x_1x_2 учитывает взаимодействие признаков
Число столбцов с константой: $m = C(p + d, d)$

Исходных признаков p	Степень d	Столбцов m
2	2	6
10	3	286
10	5	3003

При include_bias=False константного столбца нет.

Как найти коэффициенты

$$\mathbf{L(\beta) = \|y - \Phi\beta\|^2 \to \min}$$

$$\nabla L = 2\Phi^T(\Phi\beta - y) = 0$$

$$\mathbf{\Phi^T\Phi\beta = \Phi^T y}$$

При полном столбцовом ранге: $\hat{\beta} = (\Phi^T\Phi)^{-1}\Phi^T y$
В коде: np.linalg.lstsq(Φ, y, rcond=None)

Обратная матрица нужна для вывода формулы. В вычислениях используем lstsq.

Пример на пяти точках

x	y	1	x^2
−2	5	1	4
−1	2	1	1
0	1	1	0
1	2	1	1
2	6	1	4

Степень d = 2

$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$

5 наблюдений
3 коэффициента

Пять условных наблюдений без физических единиц.

Составляем нормальную систему

$\Sigma x = 0$ $\Sigma x^2 = 10$ $\Sigma x^3 = 0$ $\Sigma x^4 = 34$

$\Sigma y = 16$ $\Sigma xy = 2$ $\Sigma x^2 y = 48$

$\Phi^T\Phi$	β_0	β_1	β_2
строка 1	5	0	10
строка 2	0	10	0
строка 3	10	0	34

$\Phi^T y = (16, 2, 48)^T$

$\Phi^T\Phi\beta = \Phi^T y$

Находим коэффициенты

$5\beta_0 + 10\beta_2 = 16$
$10\beta_1 = 2$
$10\beta_0 + 34\beta_2 = 48$

$\beta_1 = 1/5$
$14\beta_2 = 16$
$\beta_2 = 8/7, \quad \beta_0 = 32/35$

$\hat{y}(x) = 32/35 + (1/5)x + (8/7)x^2$

$x^* = 1{,}5: \quad \hat{y} = 0{,}9143 + 0{,}3 + 2{,}5714 = 3{,}7857$

Из третьего уравнения вычитаем удвоенное первое и находим β_2.

Проверяем остатки

x	y	ŷ	y − ŷ
-2	5	5.0857	-0.0857
-1	2	1.8571	0.1429
0	1	0.9143	0.0857
1	2	2.2571	-0.2571
2	6	5.8857	0.1143

Парабола
SSE = 0,1143
RMSE = 0,1512

Прямая: 3,2 + 0,2x
SSE = 18,4
RMSE = 1,9183

Это ошибки на обучении. RMSE = √(SSE / n). Качество на новых данных ещё не проверено.

Тот же расчёт в Python

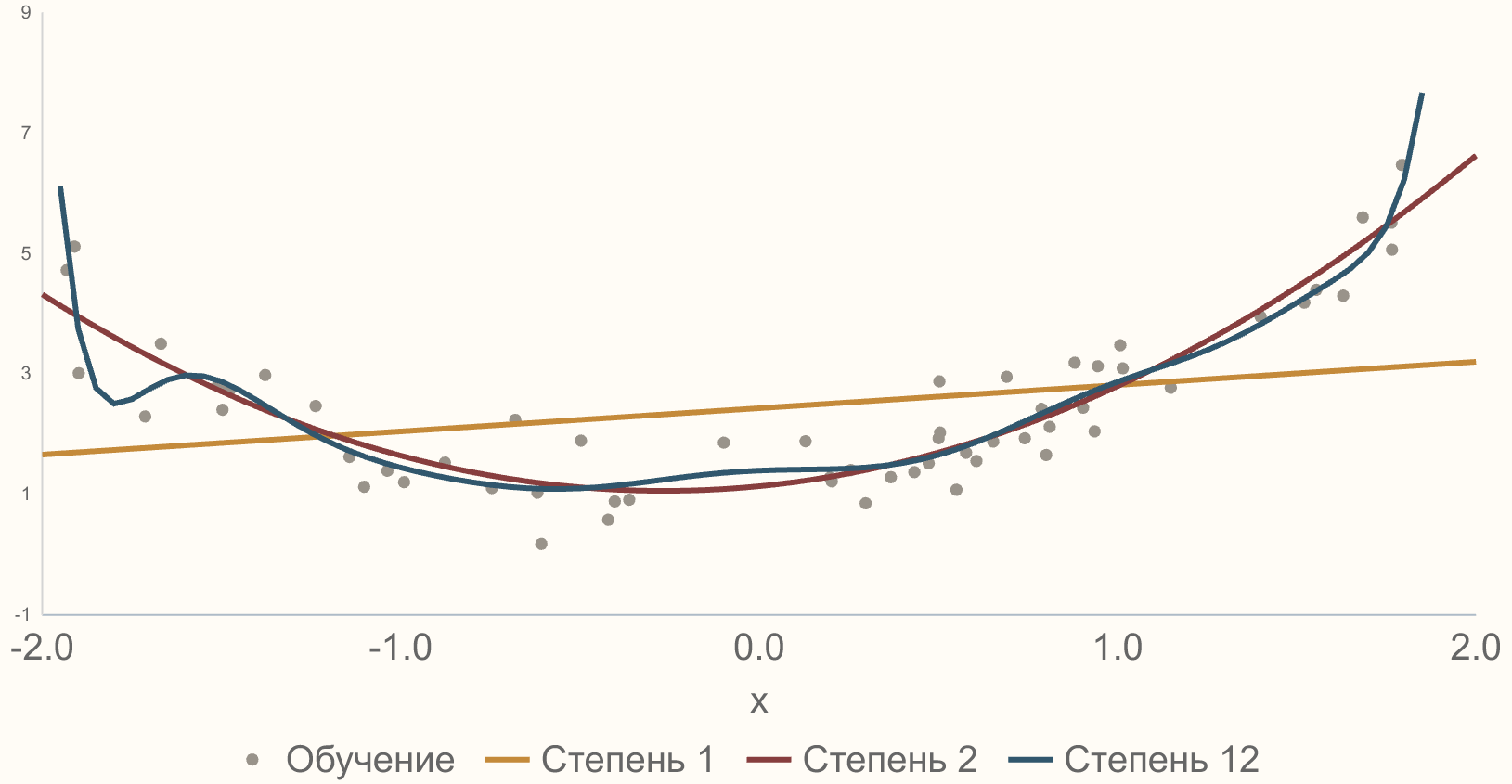
```
import numpy as np

x = np.array([-2., -1., 0., 1., 2.])
y = np.array([5., 2., 1., 2., 6.])
Phi = np.vander(x, N=3, increasing=True)
beta, *_ = np.linalg.lstsq(Phi, y, rcond=None)
pred = Phi @ beta
rmse = np.sqrt(np.mean((y - pred)**2))

# beta = [0.914286, 0.2, 1.142857]
# rmse = 0.151186
```

Код с проверкой коэффициентов: src/numerical_example.py

Выбор степени



d = 1
не описывает кривизну

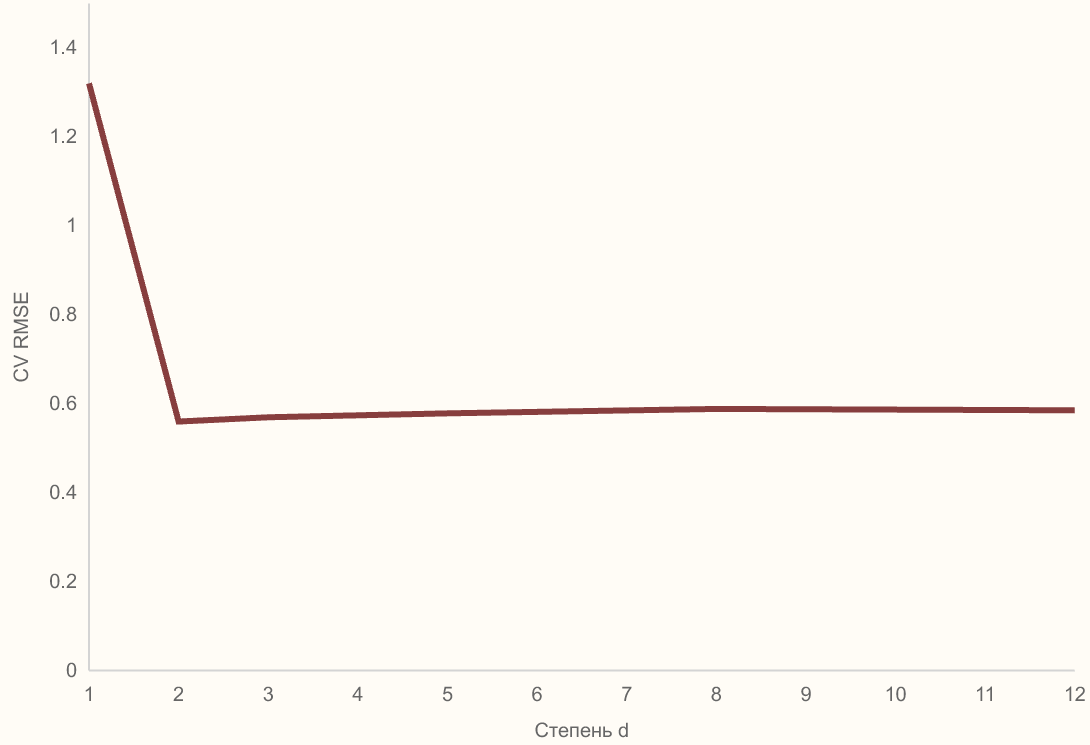
d = 2
описывает зависимость

d = 12
сильнее реагирует на шум

МНК без штрафа. Части кривых вне шкалы [−1, 9] не показаны. Полные значения: results/.

Проверка на новых данных

1. Откладываем 20 объектов для теста
2. На 60 объектах проводим 5-fold CV
3. Обучаем выбранную модель на train
4. Считаем ошибку на тесте



Масштабирование обучаем заново внутри каждого разбиения

CV RMSE = √(среднее MSE по частям). Для каждой степени выбран лучший alpha из сетки.

Масштабирование и Ridge

$$L(\beta) = \|y - \Phi\beta\|^2 + \alpha \sum_{j=1}^{m} \beta_j^2$$

Степенные признаки имеют разные масштабы
StandardScaler стандартизирует каждый столбец

Ridge ограничивает величину коэффициентов
Свободный член оставляем без штрафа

Степень d и alpha подбираем вместе

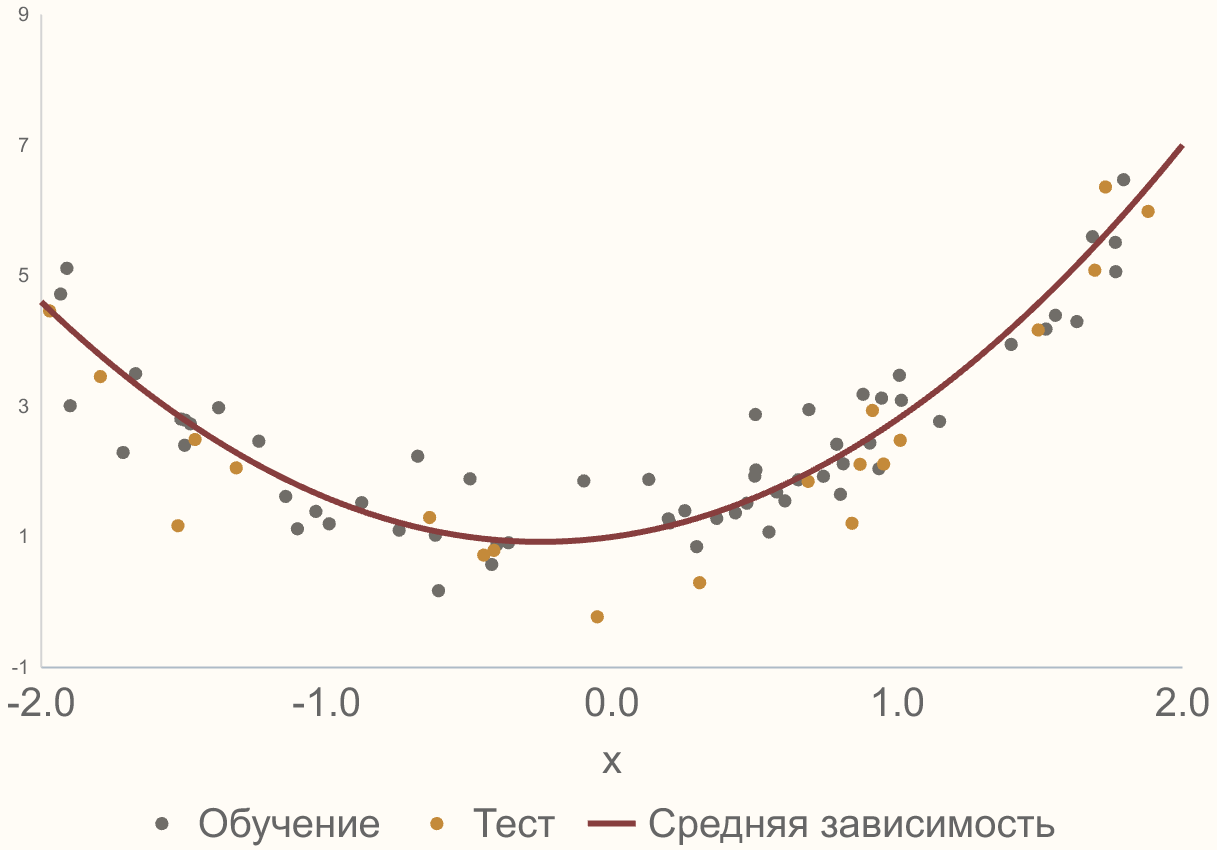
В sklearn.Ridge используется сумма квадратов остатков. Масштаб признаков влияет на штраф.

Данные для эксперимента

$y = 1 + 0{,}6x + 1{,}2x^2 + \varepsilon$
$\varepsilon \sim N(0, 0{,}6^2)$

80 наблюдений
$x \sim \text{Uniform}(-2, 2)$

60 train / 20 test
seed = 18



Синтетические данные: известна средняя зависимость, к наблюдениям добавлен шум.

Модель в scikit-learn

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
    PolynomialFeatures, StandardScaler)
from sklearn.linear_model import Ridge

pipe = Pipeline([
    ("poly", PolynomialFeatures(
        include_bias=False)),
    ("scale", StandardScaler()),
    ("ridge", Ridge()),
])
```

Константу оценивает Ridge. Его коэффициенты относятся к стандартизированным признакам.

Перебор степени и alpha

```
grid = {
    "poly__degree": [1, 2, 3, 5, 8, 12],
    "ridge__alpha": [.001, .01, .1, 1., 10.],
}
cv = KFold(5, shuffle=True, random_state=18)
search = GridSearchCV(
    pipe, grid, cv=cv,
    scoring="neg_mean_squared_error")
search.fit(X_train, y_train)
pred = search.predict(X_test)
```

Полный код и импорты: src/experiment.py. GridSearchCV использует refit=True.

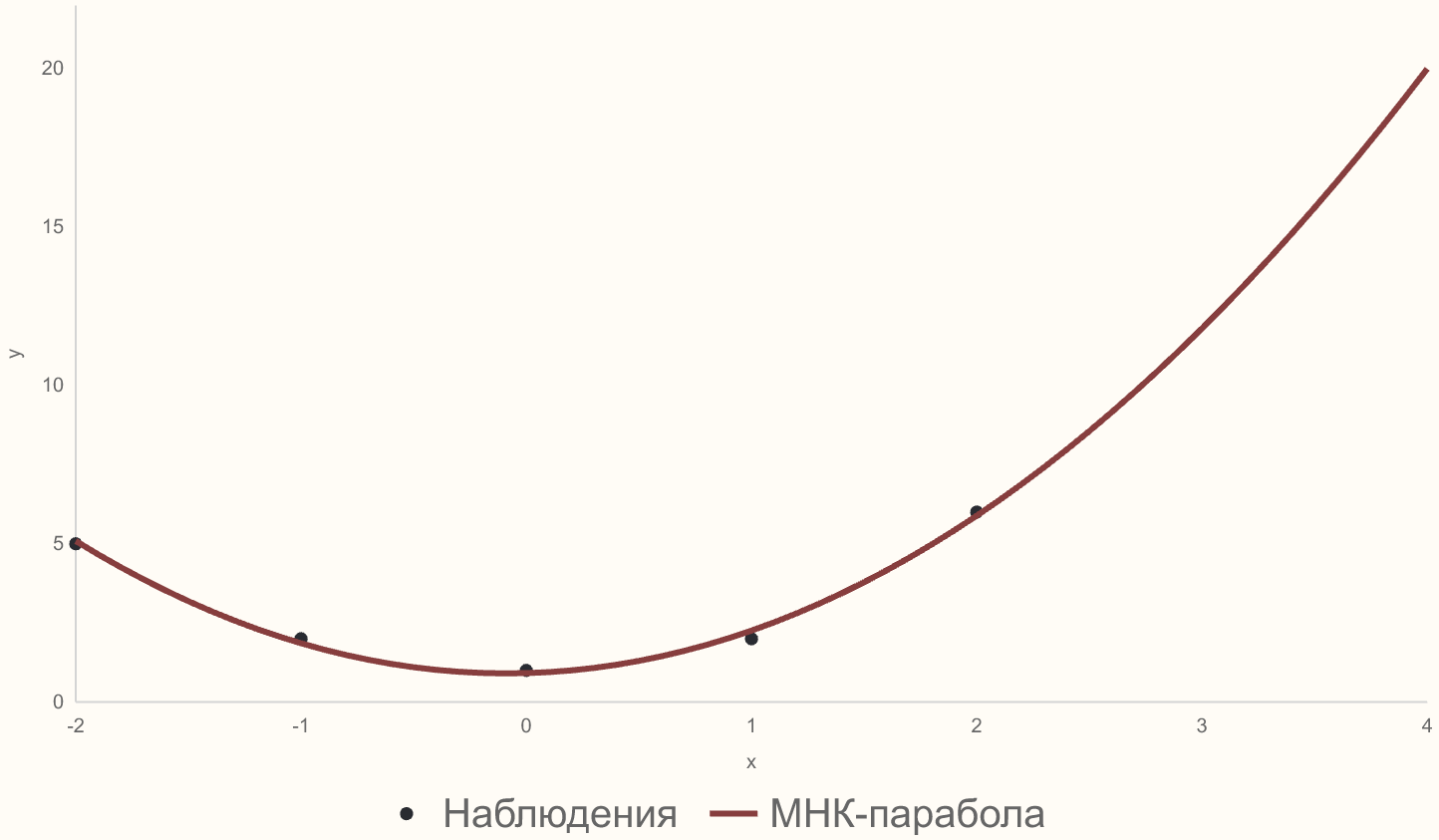
Сравнение ошибок

Модель	CV RMSE	Test RMSE	Test R^2
Линейная	1.322	1.688	0.129
Полином + Ridge	0.560	0.681	0.858
Кубический сплайн	0.587	0.686	0.856
Дерево	0.733	0.821	0.794
SVR с RBF	0.591	0.703	0.849

Полином + Ridge: d = 2, alpha = 1

Для всех моделей одинаковые train/test и CV-разбиения. Параметры подбирались только на train.

Прогноз вне диапазона данных



Исходные x: от −2 до 2

Подставим x = 4:
$\hat{y} = 20$

За этим диапазоном наблюдений нет

Тест внутри исходного диапазона не проверяет качество экстраполяции.

Чем заменить полином

Метод	Форма зависимости	Когда полезен
Линейная регрессия	Прямая / гиперплоскость	Слабая кривизна
Полином	Глобальная гладкая	Мало признаков, умеренный d
Кубический сплайн	Локальные полиномы	Меняющаяся локальная форма
Дерево	Кусочно-постоянная	Пороговые эффекты
SVR с RBF	Гибкая гладкая	Сложная форма, настройка ядра

Порядок методов по качеству может измениться на другом наборе данных.

Порядок работы

1. Отложить тест и обучить обычную прямую
2. Проверить полиномы нескольких степеней
3. Подобрать d и alpha на обучающей части
4. Посмотреть тестовую ошибку и остатки
5. Сохранить код и параметры эксперимента

Что получилось

Степенные признаки и линейное обучение коэффициентов

На пяти точках получили коэффициенты вручную

На шумных данных выбрали степень по CV

Для устойчивости использовали масштабирование и Ridge

Прогноз вне диапазона требует отдельной проверки